Relations-Foreign Key-
Data manipulation language (DML)

Agenda

- Relationships
- Degree of Relationships ?
- Foreign Key Relationships in PostgreSQL
- Foreign Key Constraints (CASCADE, SET NULL, RESTRICT)
- Data manipulation language (DML) (Insert-Update)(Delete-Select into next slide)

Relationship

- It describes the association between two entities.
- Example: The relationship between two entities Student and Class has many to many relationship.

Degree of Relationships

There are 3 types of degrees of relationship:

- Unary relationship
- Binary relationship
- Ternary relationship

Unary Relationship

Definition:
A **Unary Relationship** occurs when **one entity is related to another instance of the same entity type**.
Example:
An **EMPLOYEE** can supervise another **EMPLOYEE**.

Example scenario:
One employee is a **manager**
Other employees **report to that manager**
This means the relationship happens **within the same table**.

Unary Relationship

Example Table

EID	Name	Super-EID
1	Ahmed	NULL
2	Sara	1
3	Ali	1

Binary Relationship

Entities of two different types related to each other.

Teacher — Teaches — Subject

Example
A **Teacher** teaches a **Subject**.
Entities involved:
Teacher
Subject
Relationship:
Teaches

Binary Relationship

Example Table

TeacherID	SubjectID
1	S101
2	S102
1	S103

Explanation:
Teacher **1** teaches **Subject S101**
Teacher **2** teaches **Subject S102**
Teacher **1** also teaches **Subject S103**

Ternary Relationship

Definition:
A **Ternary Relationship** occurs when **three different entities participate in the same relationship.**
This means the relationship connects **three tables at the same time**.

Example
A **Teacher** teaches a **Subject** to a **Student**.
Example scenario:
A teacher teaches **Math**
A specific student attends that class
So the relationship involves:
Teacher
Subject
Student
All three entities are required to describe the relationship.

Ternary Relationship

Example Table

TeacherID	SubjectID	StudentID
1	101	201
2	102	202

Explanation:
Teacher **1** teaches **Subject 101** to **Student 201**
Teacher **2** teaches **Subject 102** to **Student 202**

Cardinality of Relationships

- Cardinality of a relationship can be defined as the number of times an entity of an entity set participates in a relationship set.
- Cardinality shows how the entities are connected to each other.

Types of cardinalities between tables are:

- One-to-One
- One-to-Many
- Many-to-Many

One-to-One Relationship

A **One-to-One (1:1) relationship** is a type of binary relationship where **each entity in one set is associated with exactly one entity in the other set**, and vice versa.

Example:
Person — 1 — Has — 1 — Passport

Each person has **exactly one passport**.
Each passport belongs to **exactly one person**.

One-to-Many Relationship

An entity in set A can be linked with any number of entities in set B while an entity in set B can be linked with only one entity in set A.

Example:
Customer — 1 — Places — N — Order

A single customer can place **many orders**.
Each order is placed by **only one customer**.

Many-to-Many Relationship

An entity in set A can link to any number of entities in set B while an entity in set B can link to any number of entities in set A.

Example:
Customer — M — Buys — N — Product

A single customer can buy **many products**.
A single product can be bought by **many customers**

What is Relational Mapping / ORM?

Definition:
ORM (Object-Relational Mapping) is a technique that maps database tables and relationships to objects in code, so developers can interact with data as objects instead of writing SQL queries.

- Tables → Classes / Objects
- Rows → Object Instances
- Columns → Object Attributes
- Relationships → Object References / Lists / Collections

How It Works in PostgreSQL

How It Works in PostgreSQL

PostgreSQL stores data in tables with rows and columns

Each table becomes an Object Class in the code

Foreign keys represent relationships as object references

Example:

Table	ORM Object	Relationship
EMPLOYEE	Employee Class	N:1 → Department
DEPARTMENT	Department Class	1:N → Employees
PROJECT	Project Class	M:N → Employees (WORKS_ON junction table)

CRUD via ORM:

- Create → $employee = new Employee(); $employee->save();
- Read → $employee = Employee::find(1);
- Update → $employee->salary = 6000; $employee->save();
- Delete → $employee->delete();

How It Works in ServiceNow

- ServiceNow stores data in PostgreSQL tables (or similar relational DB)

- Tables are exposed as Objects / Records in ServiceNow

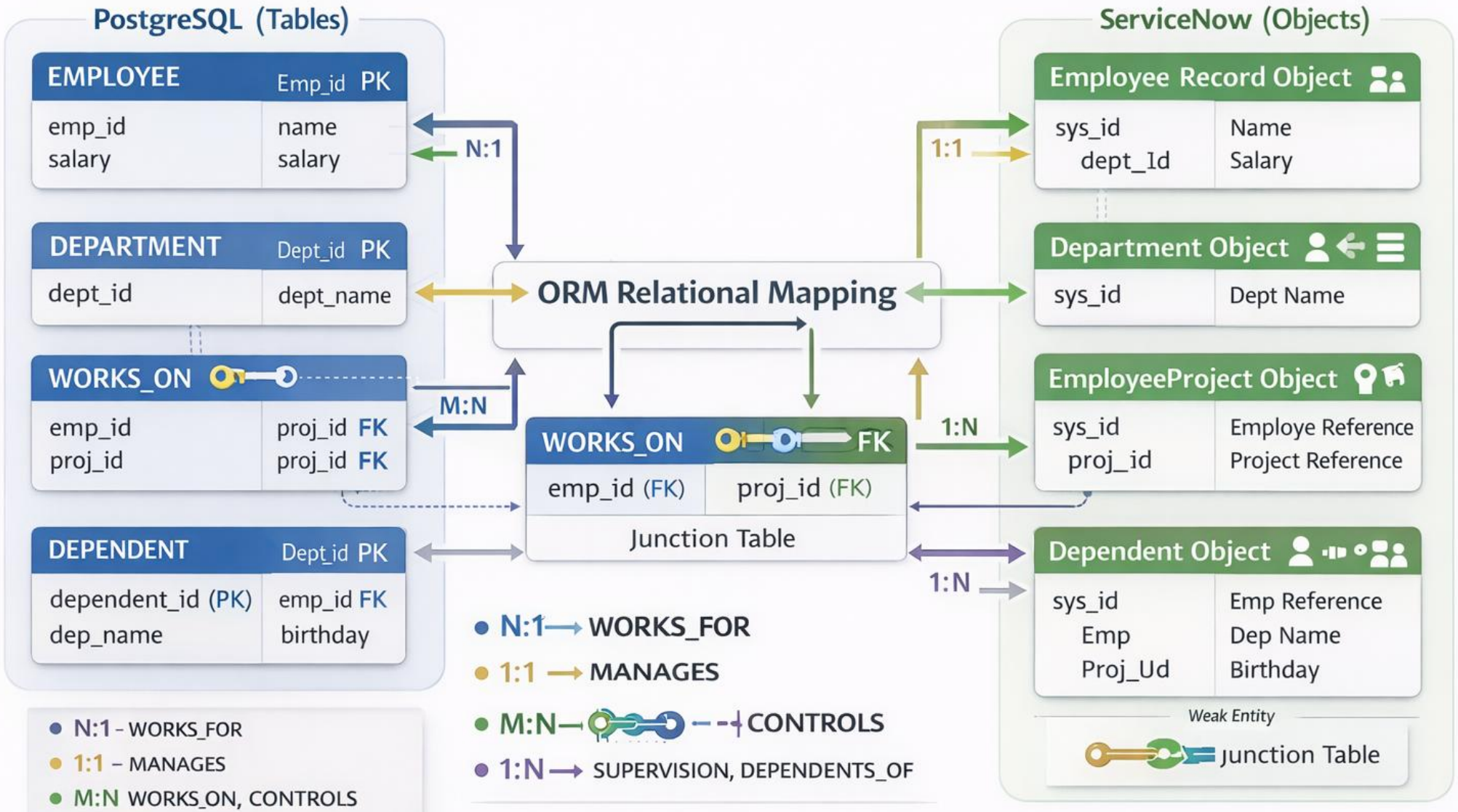
- Developers use GlideRecord API or Scripts to access these objects instead of SQL

Relationships in ServiceNow:

- 1:N → Reference fields + **related lists**
- M:N → Many-to-many relationship tables (**junction tables**)
- 1:1 → **Unique reference field**

- Tables in PostgreSQL → mapped to objects in code
- Relationships (1:1, 1:N, M:N) → handled automatically by object references / lists
- ServiceNow exposes these objects as records with APIs

ORM Relational Mapping in COMPANY Database



Foreign Key & Actions in PostgreSQL

Foreign Key

A Foreign Key (FK) is a column in a table that references the Primary Key (PK) of another table to enforce referential integrity.

When defining an FK, you can specify actions for what happens when the referenced row is updated or deleted.

Actions in PostgreSQL

CASCADE

Definition:
When a referenced row is deleted or updated, the change is automatically applied to the dependent rows.

Example:

```
CREATE TABLE Department (
    dept_id SERIAL PRIMARY KEY,
    name VARCHAR(50)
);

CREATE TABLE Employee (
    emp_id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT REFERENCES Department(dept_id) ON DELETE CASCADE
);
```

RESTRICT

Definition:
Prevents deletion or update of a referenced row if dependent rows exist.
Example:

```
CREATE TABLE Employee (
    emp_id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT REFERENCES Department(dept_id) ON DELETE RESTRICT
);
```

SET NULL

Definition:
When the referenced row is deleted/updated, the FK in dependent rows is set to NULL.

Example:

```
CREATE TABLE Employee (
    emp_id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT REFERENCES Department(dept_id) ON DELETE SET NULL
);
```

NO ACTION

Definition:
No automatic change; the DB checks referential integrity at the end of the transaction. Similar to RESTRICT, but enforcement is deferred until the end of transaction.

Example:

```
CREATE TABLE Employee (
    emp_id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT REFERENCES Department(dept_id) ON DELETE NO ACTION
);
```

CASCADE VS. NO ACTION

Rule	What Happens
CASCADE	Deletes child records automatically
NO ACTION	Prevents deletion if child records exist

NOT NULL (Constraint)

Definition:
Ensures FK column cannot be NULL. Often combined with FK constraints.

Example:

```
CREATE TABLE Employee (
    emp_id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT NOT NULL REFERENCES Department(dept_id)
);
```

Data manipulation language (DML)

Data manipulation language (DML)

- Data manipulation language (DML) provides operations that handle user requests, offering a way to access and manipulate the data that users store within a database.

- Its common functions include inserting, updating and retrieving data from the database.

DML statements

- INSERT: Adds new data to the existing database table
- UPDATE: Changes or updates values in the table
- DELETE: Removes records or rows from the table
- SELECT: Retrieves data from the table or multiple tables

DEFAULT Example

- DEFAULT on CREATE TABLE

```
CREATE TABLE Persons (
ID integer NOT NULL,
FirstName character varying(50) NOT NULL,
LastName character varying(50) NOT NULL,
Age integer,
City character varying(100) DEFAULT 'Sandnes'
);
```

INSERT INTO Statement

- The INSERT INTO statement is used to insert new records in a table.

- Syntax

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

- Example

```
INSERT INTO Persons (ID,FirstName,LastName,Age,City)
VALUES (1,'Alfred','Schmidt',20,'Norway');
```

UPDATE Statement

- The UPDATE statement is used to modify the existing records in a table.

- Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

- Example

```
UPDATE Persons SET City='Frankfurt'
WHERE ID=1;
```

Thank you